UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

**Calidad de Software · Unidad II — ISTQB · TMMi · TQM**

# Reporte Semanal de Avance

## Sprint 2 — Cancelación y Reprogramación

**Proyecto HELIOS**

Sistema unificado de citas médicas (red de policlínicas)

**Grupo HELIOS**

José Brown — Product Owner
Ethan González — Desarrollador Frontend
Alejandro Saenz — Tech Lead / Backend
Ashley Martínez — QA Engineer
Marian Barba — Scrum Master / DevOps

Sprint 2 (HELL-3) · Junio 2026
Repositorio: github.com/MarianC-programing/Helios
Producción: helios-web-production.up.railway.app

# 1. Daily Scrum — Las 3 preguntas (con evidencias)

Respuestas concretas y verificables del avance desde el laboratorio anterior.

### ¿Qué hicimos desde el último laboratorio?

- **Cancelar cita** (autoservicio del paciente): función SQL atómica + UI con modal. Commit `e68a872`.
- **Reprogramar cita** a otro horario del mismo médico: función SQL atómica + modal con calendario. Commit `e68a872`.
- **Tabla de auditoría** `citas_historial` + triggers (registra cada cambio).
- **6 bugs resueltos** (B-21 a B-26): calendario/zona horaria, cédula 5 tipos, doble confirmación, validaciones, slots vencidos, y un **hueco de seguridad RLS**.
- **Refuerzo de seguridad** (B-26): cerrado el acceso a datos ajenos en 7 tablas. Commit `3b08751`.
- **Validador de cédula a prueba de futuro** (comarcas nuevas). Commit `c1271e0`.
- **Evidencia:** 117 tests automatizados pasando, build limpio, todo desplegado en producción (HTTP 200).

### ¿Qué haremos para el próximo laboratorio?

| Actividad | Responsable | Meta |
|---|---|---|
| Sprint 3 — Sincronización entre sedes (HELL-4) | Alejandro + Ethan | Compartir disponibilidad entre policlínicas |
| Pruebas de seguridad OWASP ZAP (Sprint 5) | Ashley / Marian | Reporte de vulnerabilidades |
| Pruebas de rendimiento Lighthouse (Sprint 5) | Ashley | Core Web Vitals |
| Mover `chequeo_cedula` a RPC (deuda técnica) | Alejandro | Cerrar read-leak menor |

### ¿Qué problemas o bloqueos tuvimos?

- **Bug crítico (UNIQUE):** al cancelar, el horario quedaba inutilizable por una restricción `UNIQUE` total. **Detectado por el code review** antes del deploy y resuelto con un índice único parcial.
- **Hueco de seguridad heredado:** la policy permisiva `sprint1_all` seguía activa en 7 tablas; lo cerramos con una migración transaccional sin pérdida de datos.
- **Dependencia externa:** el proyecto Supabase pertenece a la cuenta del equipo; las migraciones SQL las ejecuta el PO en el dashboard (no automatizable desde el repo).
- **Cero regresiones:** los 65 tests del Sprint 1 siguen pasando.

---

# 2. Avance del desarrollo (evidencias técnicas)

### Código fuente

- **Frontend (React 19 + TypeScript):** hooks `useCancelarCita`, `useReprogramarCita`; componentes `BotonCancelarCita`, `ReprogramarCitaModal`; página `MisCitas` actualizada.
- **Backend (PostgreSQL / Supabase):** funciones `cancelar_cita_atomica`, `reprogramar_cita_atomica`, tabla `citas_historial` + trigger, migración de seguridad `sprint2_b26_cerrar_sprint1_all.sql`.
- Repositorio: `github.com/MarianC-programing/Helios`, rama `sprint1/bootstrap`. Commits firmados y trazables.

### Prototipo funcional (en producción)

- App en vivo: `helios-web-production.up.railway.app` — agendar, cancelar y reprogramar citas funcionando.

### Diagramas

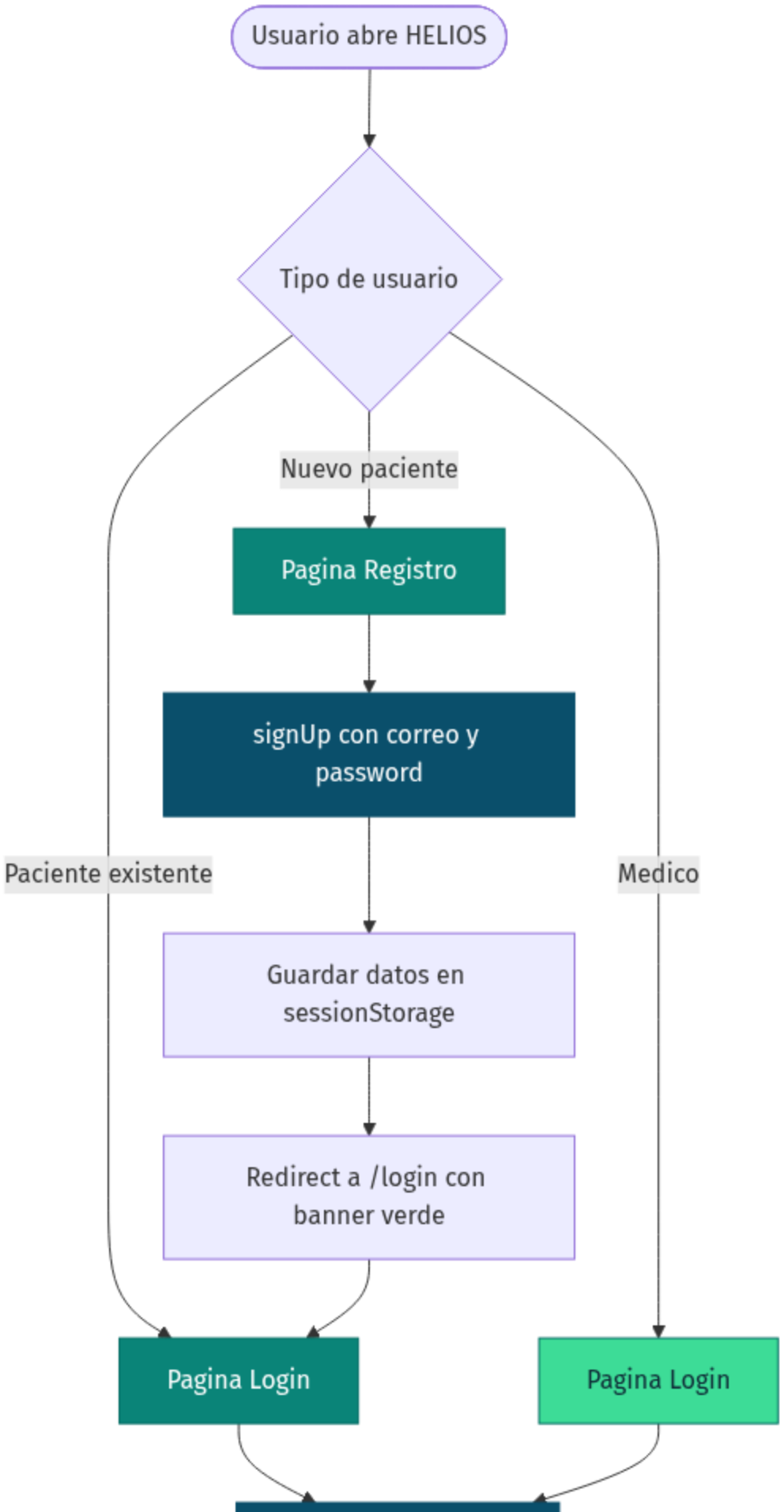

Usuario abre HELIOS
Tipo de usuario
Nuevo paciente
Pagina Registro
signUp con correo y password
Paciente existente
Medico
Guardar datos en sessionStorage
Redirect a /login con banner verde
Pagina Login
Pagina Login

Figura 1. Diagrama de flujo del usuario (paciente y médico) end-to-end.

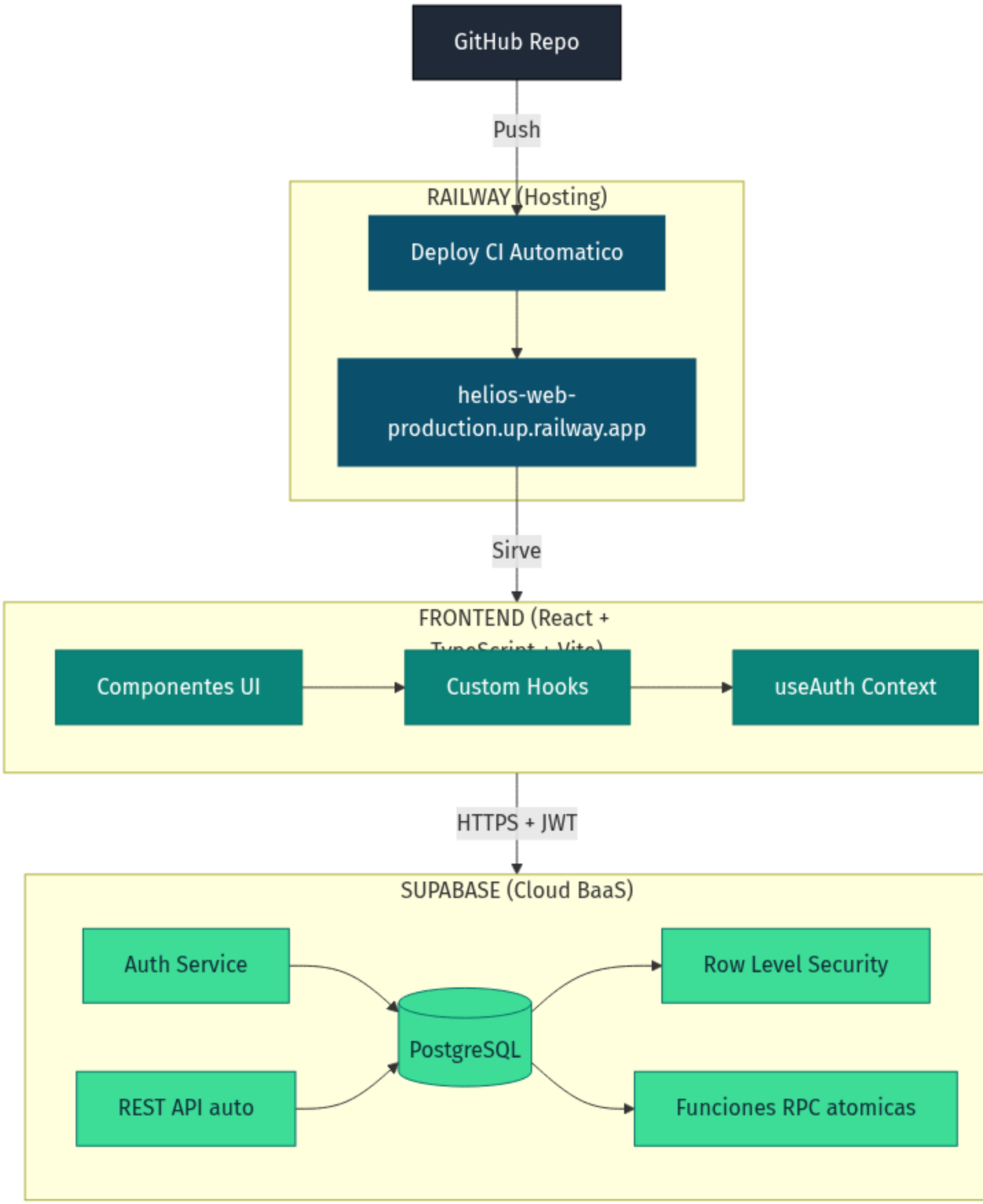

GitHub Repo
Push
RAILWAY (Hosting)
Deploy CI Automatico
helios-web-production.up.railway.app
Sirve
FRONTEND (React +
Componentes UI
Custom Hooks
useAuth Context
HTTPS + JWT
SUPABASE (Cloud BaaS)
Auth Service
Row Level Security
PostgreSQL
REST API auto
Funciones RPC atomicas

Figura 2. Arquitectura: GitHub → Railway → Frontend React → Supabase (BD + Auth + RLS).

## Pruebas realizadas

- **117 tests automatizados** (Vitest + React Testing Library), todos pasando.
- **Pruebas de carga** (Locust): P95 = 300 ms con 200 usuarios concurrentes.
- **Code review** automatizado + humano antes de cada merge (detectó el bug crítico del UNIQUE).

---

# 3. Evidencias de medición aplicadas (Unidad II)

### ISTQB — Métricas de pruebas, defectos, cobertura, esfuerzo

- **Qué medimos:** tests automatizados, cobertura de código, defectos y latencia.
- **Cómo lo medimos:** Vitest (`pnpm vitest run --coverage`) para tests/cobertura; GitHub para defectos trazables; Locust para latencia.
- **Resultados e interpretación:** 117 tests (100% pasando), 61.4% de cobertura en sentencias (rango profesional 60-80%), 6 defectos detectados y resueltos (tasa ~8.7 bugs/KLOC, normal en MVP), P95 300 ms (meta 2 s, superada 6x). Interpretación: el producto es estable y la cobertura cubre la lógica crítica.

### TMMi — Madurez del proceso de pruebas (Nivel 2: Gestionado)

- **Práctica de medición:** métricas registradas por sprint (este reporte).
- **Control:** code review obligatorio antes de merge + threshold de cobertura ≥60% en CI (el build falla si baja).
- **Mejora:** cada bug genera un test de regresión (plan de pruebas trazable). Estamos en **Nivel 2 (Gestionado)**, en camino a Nivel 3.

### TQM — Mejora continua, calidad del proceso y del producto

- **Proceso:** Plan Mode obligatorio antes de codificar; documentación viva (CLAUDE.md).
- **Producto:** el PO prueba el deploy real antes de cada entrega; los bugs detectados en uso real se corrigen en la misma jornada.
- **Evidencia de mejora:** el bug crítico del UNIQUE fue cazado por el code review y corregido ANTES de producción; cero regresiones del Sprint 1.

---

# 4. Registro de métricas

| Categoría | Valor (Sprint 2) |
|---|---|
| **Requerimientos** | HELL-3 (épica) con 13 subtareas; cancelar + reprogramar al 100% |
| **Diseño** | 2 funciones SQL atómicas nuevas + 1 tabla de auditoría + 4 componentes/hooks frontend |
| **Código** | ~2.6 KLOC totales; 11 archivos nuevos/modificados este sprint; commits firmados |
| **Defectos** | 6 detectados (B-21..B-26) · 6 resueltos · 0 pendientes · ~8.7 bugs/KLOC |
| **Esfuerzo** | Sprint de ~2 semanas; ciclos Plan→Code→Review→Test→Deploy de <30 min por bug |
| **Productividad** | ~2 deploys/día; bundle reducido 754 → 503 KB (-34%) |
| **Calidad del proceso** | Code review obligatorio · threshold CI ≥60% · Plan Mode |
| **Calidad del producto** | 117 tests pasando · P95 300 ms · seguridad RLS reforzada · 0 regresiones |

# 5. Evidencias de uso de herramientas

| Herramienta | Uso en el Sprint 2 | Evidencia |
|---|---|---|
| **GitHub** | Control de versiones, commits firmados, code review | Commits `e68a872`, `3b08751`, `c1271e0` en `MarianC-programing/Helios` |
| **Jira** | Tablero Kanban, épica HELL-3 + 13 subtareas | `helios-group.atlassian.net` (HELL-3) |
| **Vitest + RTL** | Pruebas unitarias e integración | `pnpm vitest run` → 117/117 |
| **Locust** | Pruebas de carga | P95 300 ms con 200 usuarios |
| **Supabase** | BD PostgreSQL + Auth + RLS + funciones SQL | Dashboard del proyecto HELIOS |
| **Railway** | Despliegue continuo (CI) a producción | helios-web-production.up.railway.app |
| **SonarCloud** | Análisis estático (configurado) | `sonar-project.properties` + workflow GitHub Actions |
| **OWASP ZAP / Lighthouse** | Seguridad + rendimiento (planificado Sprint 5) | Guía en DOCUMENTACION_SPRINT2.pdf |

> *Nota de honestidad: OWASP ZAP y Lighthouse están planificados para el Sprint 5 ("Pruebas y cierre"); en este sprint la calidad se midió con Vitest, Locust y code review.*

---

# 6. Actualización del backlog / Tablero Kanban (Jira HELL-3)

### Épica del Sprint 2: HELL-3 — Cancelación y Reprogramación

| Estado | Subtareas |
|---|---|
| **Done (completadas)** | B-21 calendario, B-22 cédula, B-23 doble confirmación, B-24 validaciones, B-25 slots vencidos, B-26 seguridad, SQL cancelar, SQL reprogramar, tabla historial, hooks, componentes, MisCitas, dashboard médico |
| **In Progress** | (Sprint 2 esencialmente cerrado) |
| **To Do (Sprint 5)** | Pruebas OWASP ZAP, Pruebas Lighthouse |

### Otros sprints en el tablero (contexto)

- HELL-2 Sprint 1 — Agendamiento (entregado)
- **HELL-3 Sprint 2 — Cancelación y Reprogramación (este sprint)**
- HELL-4 Sprint 3 — Sincronización entre sedes (siguiente)
- HELL-5 Sprint 4 — Notificaciones
- HELL-6 Sprint 5 — Pruebas y cierre

### Bloqueos / dependencias

- Las migraciones SQL dependen de ejecución manual del PO en Supabase (proyecto del equipo).
- ZAP/Lighthouse dependen del Sprint 5.

---

# 7. Conclusiones

El Sprint 2 cumplió su objetivo: los pacientes ya pueden **cancelar y reprogramar** sus citas de forma autoservicio, con seguridad reforzada a nivel de base de datos y auditoría de cambios. Se resolvieron 6 defectos detectados en uso real, manteniendo cero regresiones y 117 pruebas automatizadas. La aplicación de ISTQB, TMMi y TQM se evidencia en métricas reales, code review obligatorio y mejora continua (el bug crítico se cazó antes de producción).

**Próximo sprint:** Sincronización entre sedes (HELL-4).

---

## Anexo — Cómo verificar (reproducibilidad)

```
cd helios-repo/frontend
pnpm install
pnpm vitest run            # 117/117 tests
pnpm build                 # build limpio (~503 KB)
curl -sf https://helios-web-production.up.railway.app/   # HTTP 200
```

*Reporte generado para el laboratorio de Calidad de Software · UTP 2026.*